

User Manual

FSOC Coarse PAT Simulator

Installation, Operation & Configuration Guide

Version 1.0 | September 2026

Abstract

This manual explains how to install, run, configure, and interpret the FSOC Coarse PAT Simulator, a desktop application that simulates the Pointing, Acquisition, and Tracking (PAT) subsystem of a Free-Space Optical Communication terminal. It covers system requirements, installation, the three application modes (GUI dashboard, CLI preview, headless), the Mission-Control interface, the JSON configuration format, the four scenario presets, benchmarking, the five-act demonstration tour, the generated reports and telemetry, and troubleshooting. Text on a light-orange background marked *Added guidance* was written to supplement the original manual; everything else follows it.

Contents

1	Introduction	3
1.1	Conventions Used in This Manual	3
1.2	Related Documents	3
2	System Requirements	3
2.1	Hardware	3
2.2	Software	3
3	Installation	3
3.1	Step 1 — Download the Project	3
3.2	Step 2 — Create a Virtual Environment (Recommended)	4
3.3	Step 3 — Install Dependencies	4
3.4	Step 4 — Verify Installation	4
3.5	Step 5 — AI Model Setup (Optional)	5
4	Quick Start	5
4.1	Launch the GUI Dashboard	5
4.2	Launch the CLI Preview	5
4.3	Launch Headless (No Display)	5
4.4	Run the Demonstration Tour	5
5	Application Modes	6
5.1	GUI Dashboard Mode (Default)	6
5.2	CLI Preview Mode	6
5.3	Headless Mode	6
5.4	Command-Line Arguments	6

6	GUI Description	6
6.1	Main Window Layout	6
6.2	Camera Viewport	7
6.3	Telemetry Strip-Charts	7
6.4	Metric Cards	8
6.5	Tactical Overview Map	8
6.6	Control Panels (Tabbed)	8
7	Parameter Configuration	9
7.1	Configuration File Format	9
7.2	Parameter Reference	10
8	Scenario Presets	12
8.1	Easy Scenario	12
8.2	UAV Scenario	12
8.3	Satellite Scenario	12
8.4	Extreme Scenario	13
8.5	Creating Custom Scenarios	13
9	Running Benchmarks	13
9.1	Multi-Algorithm Benchmark	13
9.2	Survival Envelope Analysis	14
9.3	Custom Benchmark Run	14
10	Running the Demonstration	14
10.1	Full Demonstration Tour	14
10.2	Act Descriptions	14
10.3	Demonstration Outputs	14
11	Understanding the Output Reports	15
11.1	HTML Performance Report	15
11.2	Telemetry CSV	15
11.3	JSON Summary	15
12	Troubleshooting	16
12.1	Common Issues	16
12.2	Performance Optimisation Tips	16
12.3	Getting Help	17
A	Command Quick Reference	17
B	Output Files at a Glance	17
C	Glossary	17

1 Introduction

The FSOC Coarse PAT Simulator is a desktop software application that simulates the Pointing, Acquisition, and Tracking (PAT) subsystem of a Free-Space Optical Communication (FSOC) terminal. It provides:

- A **real-time simulation** of an optical beacon being tracked in a disturbed environment.
- A **Mission-Control GUI** with a live camera viewport, telemetry strip-charts, and interactive controls.
- A **command-line interface** for headless benchmarking and automated testing.
- A **5-act demonstration tour** showcasing all system capabilities.

1.1 Conventions Used in This Manual

- Monospaced text denotes commands, file names, configuration keys, and console output.
- Commands are shown for Windows and Linux where they differ. All commands are run from the project root directory, `fsoc-pat/`, unless stated otherwise.
- Angles in configuration files are in *degrees* (parameters ending in `_deg`).
- Boxes headed *Note* restate cautions from the original manual; boxes headed *Added guidance* are supplementary.

1.2 Related Documents

The Technical Report (`docs/TECHNICAL_REPORT.md`) describes the system architecture, the algorithms (Kalman/Particle filtering, PID control, detection fusion), and the benchmark results. This manual concentrates on *operating* the software.

2 System Requirements

2.1 Hardware

Component	Minimum	Recommended
CPU	Dual-core, 2.0 GHz	Quad-core, 3.0+ GHz
RAM	4 GB	8 GB
Display	1280 × 720	1920 × 1080 or higher
Storage	500 MB free	1 GB free
GPU	Not required	NVIDIA GPU (for AI model inference)

Table 1: Hardware requirements.

2.2 Software

3 Installation

3.1 Step 1 — Download the Project

If the project was provided as a ZIP archive, extract it to your preferred directory. If using Git:

```
git clone <repository-url> fsoc-pat
cd fsoc-pat
```

Dependency	Version
Operating system	Windows 10/11 (64-bit) or Ubuntu 20.04+
Python	3.9 or later
pip	21.0 or later
Git	2.30+ (optional, for cloning)

Table 2: Software requirements.

3.2 Step 2 — Create a Virtual Environment (Recommended)

```
# Windows
python -m venv venv
venv\Scripts\activate

# Linux / macOS
python3 -m venv venv
source venv/bin/activate
```

Added guidance

The original manual lists only Windows and Ubuntu as supported operating systems (Table 2), although the virtual-environment commands above also mention macOS. Treat macOS as untested.

3.3 Step 3 — Install Dependencies

```
pip install -r requirements.txt
```

This installs the packages in Table 3.

Package	Purpose
numpy>=1.24.0	Array computation and linear algebra
opencv-python>=4.8.0	Image processing, detection, and rendering
PySide6>=6.5.0	Qt6-based GUI framework
pyqtgraph>=0.13.0	Real-time plotting and strip-charts
matplotlib>=3.7.0	Report chart generation
ultralytics>=8.0.0	AI YOLO model loading and inference
pytest>=7.0.0	Test execution framework

Table 3: Python dependencies installed from `requirements.txt`.

Note

`ultralytics` is optional. The system falls back to classical-only detection if no AI model is provided.

3.4 Step 4 — Verify Installation

Run the test suite to confirm everything is working:

```
cd fsoc-pat
python -m pytest tests/ -v
```

All 23 tests should pass.

3.5 Step 5 — AI Model Setup (Optional)

To enable AI-based beacon detection, place a trained YOLO model file in the `models/` directory:

```
fsoc-pat/
models/
  beacon_detector.onnx      <- place the model file here
```

Supported formats:

- `.onnx` — inferred via OpenCV DNN (no GPU required);
- `.pt` — inferred via Ultralytics YOLO (requires PyTorch).

If no model file is present, the system operates with classical CV detection only, which is fully functional.

4 Quick Start

4.1 Launch the GUI Dashboard

```
cd fsoc-pat
python app/main.py
```

This opens the full Mission-Control GUI with a live camera viewport, telemetry charts, and interactive disturbance controls. The simulation starts automatically.

4.2 Launch the CLI Preview

To run the simulation with an OpenCV viewport (lighter than the full GUI):

```
python app/main.py --preview
```

4.3 Launch Headless (No Display)

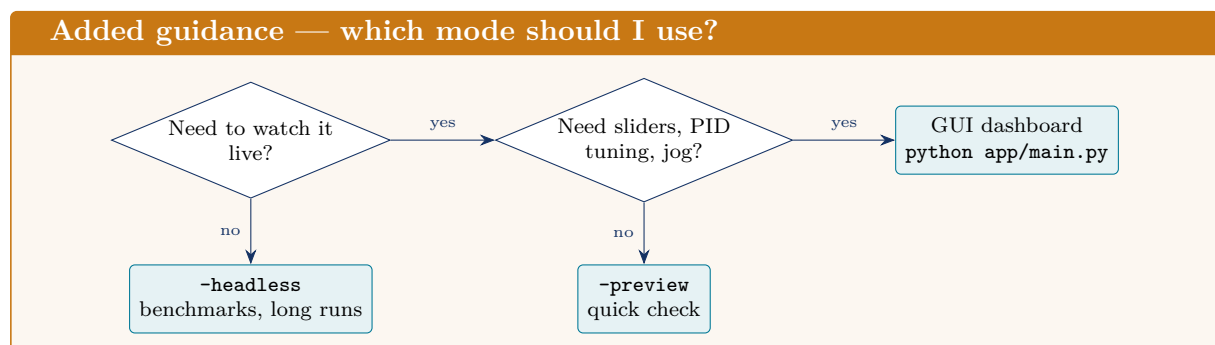
For automated benchmarking without any visual display:

```
python app/main.py --headless --duration 300
```

4.4 Run the Demonstration Tour

```
python demo.py
```

This runs a 5-act guided demonstration. Add `-preview` for a live viewport, or use headless mode (the default).



5 Application Modes

The application supports three operational modes.

5.1 GUI Dashboard Mode (Default)

Launch: `python app/main.py`

The full PySide6 Mission-Control interface, with:

- real-time camera sensor viewport with HUD overlay;
- live telemetry strip-charts (pointing error, gimbal attitude, FPS);
- interactive disturbance control sliders;
- PID gain-tuning panel;
- manual gimbal jog controls;
- scenario preset selector;
- tactical overview map.

5.2 CLI Preview Mode

Launch: `python app/main.py -preview`

An OpenCV-based live viewport with integrated HUD overlay. Suitable for quick validation without the overhead of the full Qt GUI.

5.3 Headless Mode

Launch: `python app/main.py -headless`

No display output. The simulation runs at maximum speed and generates telemetry CSV, JSON, and HTML report files. Ideal for long-duration runs and benchmark sweeps.

5.4 Command-Line Arguments

Argument	Description	Default
<code>-config <path></code>	Path to scenario JSON configuration file	<code>configs/default.json</code>
<code>-preview</code>	Show OpenCV live viewport	Off
<code>-headless</code>	Run without any display	Off
<code>-duration <seconds></code>	Simulation duration in seconds	30
<code>-output-dir <path></code>	Directory for generated reports	<code>reports/</code>

Table 4: Arguments accepted by `app/main.py`.

6 GUI Description

6.1 Main Window Layout

The GUI dashboard uses a responsive splitter-based layout with four main zones (Fig. ??).

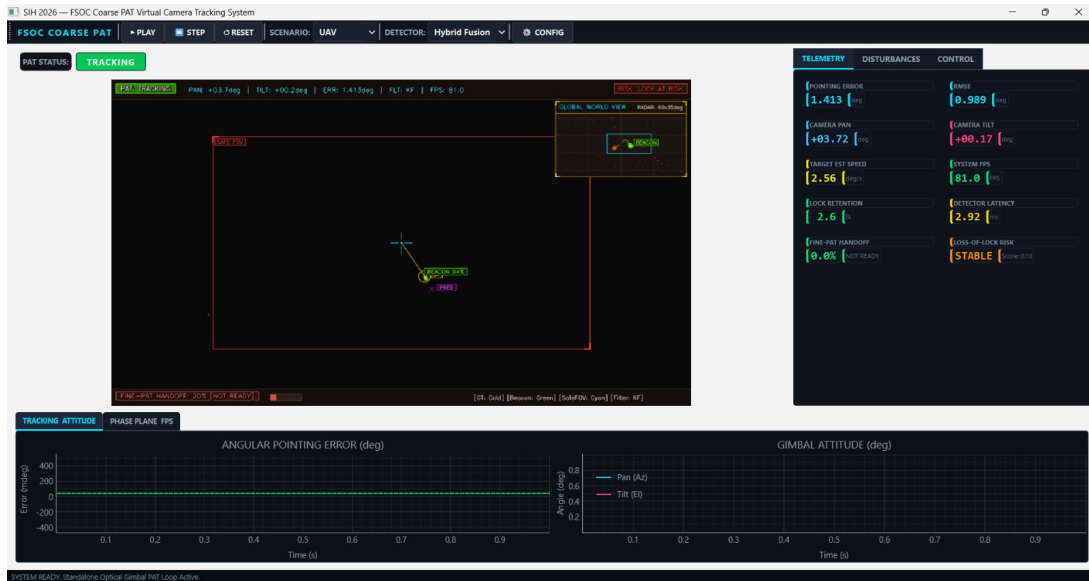


Figure 1: Main window of the Mission-Control GUI

6.2 Camera Viewport

The main viewport shows the virtual camera's sensor feed with a comprehensive HUD overlay (Table 5).

Sym.	HUD element	Description
+	Boresight crosshair	Green cross at centre indicating the camera pointing axis
○	Lock dead-zone circle	Orange circle showing the PID dead-zone region
□	Target bounding box	Cyan rectangle around the detected beacon
◆	Ground-truth marker	Red diamond at the true beacon position
●	Predicted position	Yellow dot showing the Kalman lead-ahead prediction
□	Safe FOV boundary	Dashed green/yellow/red rectangle showing the dynamic safe tracking zone
☰	Telemetry panel	Overlay with numeric read-outs for error, state, confidence, and filter mode
■	State badge	Colour-coded PAT state indicator (green = LOCKED, yellow = TRACKING, red = LOST)
▬	Handoff progress bar	Visual indicator of Fine-PAT handoff readiness percentage

Table 5: HUD overlay elements.

6.3 Telemetry Strip-Charts

Four real-time PyQtGraph charts update continuously:

1. **Pointing Error vs. Time:** radial pointing error (degrees) with a lock-threshold line.
2. **Gimbal Attitude vs. Time:** pan (azimuth) and tilt (elevation) angles with ground-truth overlay.
3. **Phase Trajectory:** 2D parametric plot of the azimuth-versus-elevation error path.
4. **System Performance:** FPS and detector latency (ms) over time.

6.4 Metric Cards

The bottom metric strip displays real-time KPIs (Table 6).

Card	Description	Colour coding
Error	Current pointing error in degrees	Green < 0.5°, Yellow < 2°, Red > 2°
State	Current PAT FSM state	Colour matches state severity
Confidence	Detector confidence score (0–100 %)	Green > 70 %, Red < 30 %
FPS	Frames per second	Green > 25, Red < 15
Handoff	Fine-PAT handoff readiness (0–100 %)	Green > 80 %, Red < 50 %

Table 6: Metric cards.

6.5 Tactical Overview Map

The lower-left panel shows a panoramic starfield map of the entire angular world (60° × 35°). It displays:

- all beacon positions with coloured trail histories;
- the camera’s current FOV footprint (green wireframe rectangle);
- beacon identification labels.

6.6 Control Panels (Tabbed)

Three tabbed panels provide interactive controls.

Tab 1: Disturbance Controls

Slider	Range	Effect
Sensor Noise	0–100 %	Gaussian pixel-noise amplitude
Vibration	0–100 %	Platform angular-jitter amplitude
Turbulence	0–100 %	Atmospheric beam-wander intensity
Motion Blur	0–100 %	Motion-blur kernel size
Frame Drop	0–100 %	Probability of dropped frames

Table 7: Disturbance sliders.

Tab 2: PID Tuning

Tab 3: Manual Jog

Directional buttons for manually commanding gimbal motion:

- **Up / Down:** tilt-axis jog;
- **Left / Right:** pan-axis jog;
- **Centre:** return the gimbal to boresight.

Parameter	Range	Description
Kp (Pan)	0.0–5.0	Proportional gain (azimuth axis)
Ki (Pan)	0.0–1.0	Integral gain (azimuth axis)
Kd (Pan)	0.0–2.0	Derivative gain (azimuth axis)
Kp (Tilt)	0.0–5.0	Proportional gain (elevation axis)
Ki (Tilt)	0.0–1.0	Integral gain (elevation axis)
Kd (Tilt)	0.0–2.0	Derivative gain (elevation axis)
Dead Zone	0.0–0.5°	Error tolerance for lock stability

Table 8: PID tuning controls.

Added guidance — a suggested first session

1. Launch the GUI and select the *Easy* preset in the toolbar; wait for the state badge to turn green (LOCKED).
2. Open the *Disturbance* tab and raise *Vibration* and *Turbulence* gradually, watching the Error and Confidence cards and the strip-charts.
3. Use *Manual Jog* to push the target off boresight, then observe reacquisition.
4. Open the *PID Tuning* tab and change one gain at a time so that its effect is visible.

7 Parameter Configuration

7.1 Configuration File Format

The system is configured with JSON files in the `configs/` directory. The structure is:

```
{
  "world": {
    "width_deg": 60.0,
    "height_deg": 35.0,
    "background_type": "starfield",
    "star_density": 0.002,
    "ambient_light": 10
  },
  "camera": {
    "resolution": [1280, 720],
    "fov_horizontal_deg": 20.0,
    "fov_vertical_deg": 11.25,
    "pan_limits_deg": [-45.0, 45.0],
    "tilt_limits_deg": [-25.0, 25.0],
    "max_pan_speed_deg_s": 25.0,
    "max_tilt_speed_deg_s": 25.0,
    "fps": 30.0
  },
  "beacon": {
    "id": 1,
    "name": "primary_optical_beacon",
    "initial_pos_deg": [2.0, 1.5],
    "intensity": 255,
    "radius_pixels": 7.0,
    "trajectory": {
      "type": "sinusoidal",
      "vx_deg_s": 1.2,
      "vy_amplitude_deg": 2.5,
      "vy_frequency_hz": 0.15
    }
  },
  "distractors": [ ... ]
}
```

```

},
"detection": {
  "detector_type": "classical",
  "binary_threshold": 160,
  "min_area": 6.0,
  "max_area": 1200.0,
  "min_circularity": 0.55
},
"pid": {
  "kp_pan": 1.1,
  "ki_pan": 0.04,
  "kd_pan": 0.18,
  "dead_zone_deg": 0.04
},
"tracking": {
  "prediction_horizon_steps": 5,
  "uncertainty_timeout_steps": 10,
  "loss_timeout_steps": 25
},
"disturbances": {
  "sensor_noise_std": 0.0,
  "vibration_amplitude_deg": 0.0,
  "turbulence_strength": 0.0,
  "motion_blur_kernel_size": 0,
  "frame_drop_prob": 0.0
}
}

```

Note

The "distractors": [...] entry is elided in this example; see configs/extreme.json for a scenario that defines distractor objects. Keys documented in the reference below (such as `kp_tilt`, `derivative_filter_alpha`, or `gate_threshold_deg`) may be added to the corresponding section when the defaults need to be changed.

7.2 Parameter Reference

World Parameters

Parameter	Type	Description
<code>width_deg</code>	float	Angular width of the world (degrees)
<code>height_deg</code>	float	Angular height of the world (degrees)
<code>background_type</code>	string	Background rendering type: "starfield" or "black"
<code>star_density</code>	float	Density of background stars (stars per pixel)
<code>ambient_light</code>	int	Background illumination level (0–255)

Parameter	Type	Description
resolution	[int, int]	Sensor resolution in pixels [width, height]
fov_horizontal_deg	float	Horizontal field of view (degrees)
fov_vertical_deg	float	Vertical field of view (degrees)
pan_limits_deg	[float, float]	Azimuth hard-stop limits [min, max]
tilt_limits_deg	[float, float]	Elevation hard-stop limits [min, max]
max_pan_speed_deg_s	float	Maximum azimuth slew rate ($^{\circ}/s$)
max_tilt_speed_deg_s	float	Maximum elevation slew rate ($^{\circ}/s$)
fps	float	Sensor frame rate (Hz)

Parameter	Type	Description
initial_pos_deg	[float, float]	Initial position [azimuth, elevation] (degrees)
intensity	int	Beacon brightness (0–255)
radius_pixels	float	Beacon PSF radius in pixels
trajectory.type	string	Motion type: "linear", "circular", "sinusoidal", "maneuver"

Camera Parameters

Beacon Parameters

PID Parameters

Tracking Parameters

Disturbance Parameters

Added guidance — percentages vs. parameters

The GUI sliders (0–100%) and the scenario descriptions in Section 8 (for example “12% noise”) are normalised intensities. The JSON file stores physical values with the ranges above (for example `turbulence_strength` in $[0, 1]$, so $25\% \approx 0.25$). The exact mapping from a percentage to `sensor_noise_std`, `vibration_amplitude_deg`, and `motion_blur_kernel_size` is defined in the software; to reproduce a preset exactly, copy its JSON file rather than converting percentages by hand.

Added guidance — general PID tuning

Change one gain at a time and observe the pointing-error chart.

- Raising **K_p** speeds up convergence but eventually causes overshoot and oscillation.
- **K_i** removes steady offsets on moving targets; too much causes slow, growing swings.
- **K_d** damps overshoot; too much amplifies pixel-quantisation noise (mitigated by the derivative filter).
- A larger **dead zone** makes the lock steadier but permits a larger residual error.

Parameter	Type	Default	Description
kp_pan	float	1.1	Proportional gain (azimuth)
ki_pan	float	0.04	Integral gain (azimuth)
kd_pan	float	0.18	Derivative gain (azimuth)
kp_tilt	float	1.1	Proportional gain (elevation)
ki_tilt	float	0.04	Integral gain (elevation)
kd_tilt	float	0.18	Derivative gain (elevation)
dead_zone_deg	float	0.04	Dead-zone (degrees)
derivative_filter_alpha	float	0.75	Low-pass filter on the D-term (0–1)

Parameter	Type	Default	Description
prediction_horizon_steps	int	5	Frames for Kalman lead-ahead prediction
uncertainty_timeout_steps	int	10	Frames before UNCERTAIN → LOST
loss_timeout_steps	int	25	Frames before a full reacquisition search is triggered
gate_threshold_deg	float	1.5	Chi-square innovation gate radius (degrees)

8 Scenario Presets

8.1 Easy Scenario

Config: `configs/easy.json`. Low-difficulty scenario for initial familiarisation.

- Target: slow circular trajectory ($\sim 1\text{r/s}$).
- Disturbances: none (clean channel).
- Expected: rapid acquisition, stable LOCKED state, near-zero error.

```
python app/main.py --config configs/easy.json
```

8.2 UAV Scenario

Config: `configs/uav.json`. Moderate scenario simulating an airborne optical terminal.

- Target: sinusoidal drift at 1.5r/s .
- Disturbances: 12 % noise, 15 % vibration, 15 % turbulence.
- Expected: tracking with occasional DEGRADED state transitions.

```
python app/main.py --config configs/uav.json
```

8.3 Satellite Scenario

Config: `configs/satellite.json`. Simulates a LEO satellite optical downlink.

- Target: linear pass at 2r/s .
- Disturbances: 8 % noise, 10 % vibration.
- Expected: continuous tracking with lead-ahead prediction active.

```
python app/main.py --config configs/satellite.json
```

Parameter	Type	Range	Description
sensor_noise_std	float	0.0–50.0	Gaussian noise standard deviation (pixel intensity)
vibration_amplitude_deg	float	0.0–2.0	Platform vibration amplitude (degrees)
vibration_frequency_hz	float	0.0–50.0	Platform vibration frequency (Hz)
turbulence_strength	float	0.0–1.0	Atmospheric turbulence intensity (normalised)
motion_blur_kernel_size	int	0–31	Motion-blur convolution kernel size (pixels)
frame_drop_prob	float	0.0–1.0	Probability of a sensor frame being dropped

Preset	Config	Target	Disturbances
Easy	easy.json	Slow circular, $\sim 1\text{r/s}$	None (clean channel)
UAV	uav.json	Sinusoidal drift, 1.5r/s	12 % noise, 15 % vibration, 15 % turbulence
Satellite	satellite.json	Linear pass, 2r/s	8 % noise, 10 % vibration
Extreme	extreme.json	Maneuver (random jinking) + distractors	20 % noise, 20 % vibration, 25 % turbulence, 15 % motion blur

Table 9: Scenario presets at a glance.

8.4 Extreme Scenario

Config: configs/extreme.json. Maximum-difficulty scenario for stress testing.

- Target: maneuver trajectory with random jinking, plus distractor objects.
- Disturbances: 20 % noise, 20 % vibration, 25 % turbulence, 15 % motion blur.
- Expected: intermittent LOST/REACQUIRING cycles; particle-filter engagement likely.

```
python app/main.py --config configs/extreme.json
```

8.5 Creating Custom Scenarios

Copy any existing configuration file and modify its parameters:

```
cp configs/default.json configs/my_scenario.json
```

Edit my_scenario.json in any text editor, then launch:

```
python app/main.py --config configs/my_scenario.json
```

9 Running Benchmarks

9.1 Multi-Algorithm Benchmark

Compare all five algorithm configurations (A through E) across scenarios:

```
python demo.py --benchmark
```

This runs each algorithm on each scenario and generates:

- per-algorithm CSV telemetry;
- a comparative JSON summary;
- an HTML performance report with charts.

9.2 Survival Envelope Analysis

Generate the 2D capability envelope matrix:

```
python -m evaluation.survival_envelope
```

Outputs:

- lock-retention matrix (target velocity vs. turbulence);
- safe operational boundary summary;
- adversarial failure points discovered.

9.3 Custom Benchmark Run

```
python app/main.py --headless --duration 120 --config configs/satellite.json --output-dir reports/satellite_test/
```

10 Running the Demonstration

10.1 Full Demonstration Tour

The 5-act demonstration showcases all system capabilities in sequence:

```
python demo.py
```

Or, with a live viewport:

```
python demo.py --preview
```

10.2 Act Descriptions

Act	Duration	Demonstrates
Act 1 — Acquisition & Search	~ 5 s	Spiral scan, candidate detection, M-of-N verification, lock convergence
Act 2 — Dynamic Tracking	~ 8 s	Sinusoidal target, Kalman prediction, lead-ahead control
Act 3 — Turbulence Survival	~ 6 s	Turbulence injection, vibration, hybrid KF↔PF, Safe-FOV risk
Act 4 — Occlusion Recovery	~ 5 s	Dynamic occluder, dead-reckoning coast, reacquisition
Act 5 — Handoff Readiness	~ 5 s	Error convergence, handoff score evaluation

Table 10: The five demonstration acts (total ≈ 29 s of simulated time).

10.3 Demonstration Outputs

Each act generates:

- `act_X_telemetry.csv` — per-frame telemetry data;
- `act_X_summary.json` — performance-summary KPIs;
- `final_performance_report.html` — comprehensive HTML report;
- console narration with real-time performance metrics.

(Here **X** stands for the act number; `final_performance_report.html` is produced once for the whole tour.)

11 Understanding the Output Reports

11.1 HTML Performance Report

Generated in the `reports/` directory after each run. It contains:

- **Executive Summary:** duration, frame count, RMSE, lock-retention rate;
- **Error Timeline Chart:** pointing error over the entire simulation;
- **Gimbal Attitude Chart:** pan/tilt angles with ground-truth overlay;
- **KPI Scorecard:** all quantitative metrics in a formatted table;
- **Per-Act Breakdown:** individual act results (demonstration mode only).

11.2 Telemetry CSV

Each row represents one simulation frame, with the columns in Table 11.

Column	Description
<code>sim_time</code>	Simulation timestamp (seconds)
<code>ground_truth_az_deg</code>	True beacon azimuth (degrees)
<code>ground_truth_el_deg</code>	True beacon elevation (degrees)
<code>camera_pan_deg</code>	Camera pan angle (degrees)
<code>camera_tilt_deg</code>	Camera tilt angle (degrees)
<code>pointing_error_deg</code>	Radial pointing error (degrees)
<code>is_locked</code>	Whether the system is in the LOCKED state
<code>pat_state</code>	Current PAT FSM state name
<code>detector_latency_ms</code>	Detection pipeline latency (ms)
<code>fps</code>	Frame rate (Hz)

Table 11: Telemetry CSV columns.

11.3 JSON Summary

A structured JSON file containing all computed performance metrics. It can be loaded programmatically for further analysis:

```
import json
with open("reports/summary.json") as f:
    metrics = json.load(f)
print(f"RMSE: {metrics['rmse_deg']:.4f} deg")
```

Added guidance — recomputing RMSE from the CSV

The pointing error in the telemetry CSV can be re-analysed with the standard library alone. The file name below is illustrative; use the CSV written by your run.

```
import csv, math
with open("reports/telemetry.csv") as f:
    err = [float(r["pointing_error_deg"]) for r in csv.DictReader(f)]
rmse = math.sqrt(sum(e * e for e in err) / len(err))
print(f"RMSE over {len(err)} frames: {rmse:.4f} deg")
```

12 Troubleshooting

12.1 Common Issues

Issue	Cause	Solution
ModuleNotFoundError: No module named 'PySide6'	Dependencies not installed	Run <code>pip install -r requirements.txt</code>
ModuleNotFoundError: No module named 'simulation'	Wrong working directory	<code>cd</code> to the <code>fsoc-pat/</code> root directory
GUI window is blank or black	GPU driver issue with Qt	Add the environment variable <code>QT_QUICK_BACKEND=software</code>
Very low FPS (< 5)	CPU-bound rendering	Use <code>-headless</code> mode or close other applications
AI detector shows “fallback mode”	No model file found	Place a <code>.onnx</code> or <code>.pt</code> file in the <code>models/</code> directory
TypeError in trajectory code	Degree/radian mismatch	Ensure the config uses degrees for all <code>_deg</code> parameters
OpenCV encoding errors on Windows	Unicode characters in output	Terminal encoding issue; ensure a UTF-8 terminal
qt.qpa.plugin: Could not find the Qt platform plugin	Qt platform missing	Install <code>libgl1-mesa-glx</code> (Linux) or reinstall PySide6

Table 12: Common issues and solutions.

Added guidance

On newer Ubuntu releases the package `libgl1-mesa-glx` is no longer available and `libgl1` is used instead. If the Qt platform-plugin error persists after installing the OpenGL library, reinstalling PySide6 (`pip install -force-reinstall PySide6`) is the other remedy listed above.

12.2 Performance Optimisation Tips

1. **Use headless mode** for batch benchmarking — it eliminates rendering overhead.
2. **Reduce camera resolution** in the config file for faster simulation.
3. **Disable the AI detector** if not needed — classical CV is significantly faster.
4. **Close other applications** to free CPU resources for the simulation loop.
5. **Lower the FPS** target in the config to reduce per-second computation.

12.3 Getting Help

- Review the demonstration tour: `python demo.py -preview`.
- Read the Technical Report: `docs/TECHNICAL_REPORT.md`.
- Run the test suite for diagnostics: `python -m pytest tests/ -v -tb=short`.
- Check the console output for [INFO], [WARN], and [ERROR] messages.

A Command Quick Reference

Command	Purpose
<code>pip install -r requirements.txt</code>	Install dependencies
<code>python -m pytest tests/ -v</code>	Run the test suite
<code>python app/main.py</code>	GUI dashboard (default scenario)
<code>python app/main.py -preview</code>	OpenCV live viewport
<code>python app/main.py -headless -duration 300</code>	Headless 300 s run
<code>python app/main.py -config configs/uav.json</code>	Run a specific scenario
<code>python demo.py</code>	Five-act demonstration
<code>python demo.py -preview</code>	Demonstration with live viewport
<code>python demo.py -benchmark</code>	Five-algorithm benchmark
<code>python -m evaluation.survival_envelope</code>	Survival-envelope matrix

Table 13: Frequently used commands.

B Output Files at a Glance

File	Produced by
<code>reports/*.html</code>	Any run (performance report)
<code>telemetry CSV</code>	Any run (per-frame data, Table 11)
<code>reports/summary.json</code>	Any run (metrics summary)
<code>act_X_telemetry.csv</code>	Demonstration tour, each act
<code>act_X_summary.json</code>	Demonstration tour, each act
<code>final_performance_report.html</code>	Demonstration tour

Table 14: Where results end up. Use `-output-dir` to redirect them.

C Glossary

PAT	Pointing, Acquisition, and Tracking.
FSOC	Free-Space Optical Communication.
FOV	Field of View of the camera sensor.
HUD	Heads-up display drawn over the camera image.

KF / PF	Kalman Filter / Particle Filter.
Dead zone	Error band inside which the PID produces no correction, stabilising the lock.
Lead-ahead prediction	Aiming at the target's predicted position several frames ahead to compensate for control latency.
Handoff	Transfer of pointing control from coarse PAT to a fine-pointing subsystem.
Headless	Running without any graphical display.